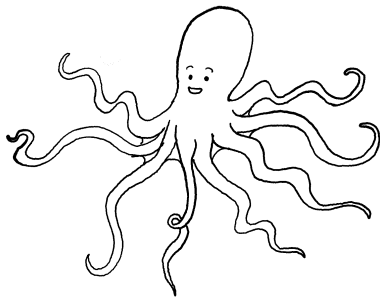


Somewhat efficient blind recommender system for the MovieLens dataset

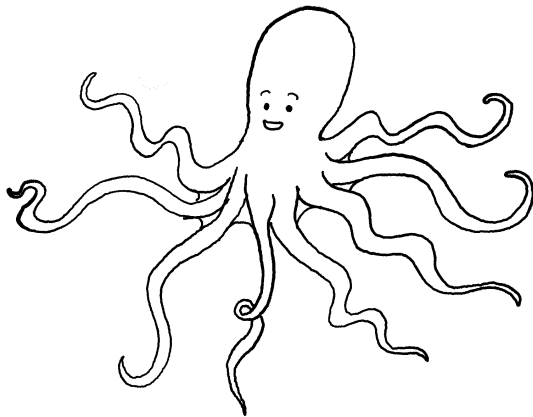
Lorenzo Rovida

Polytechnic University of Turin, Italy

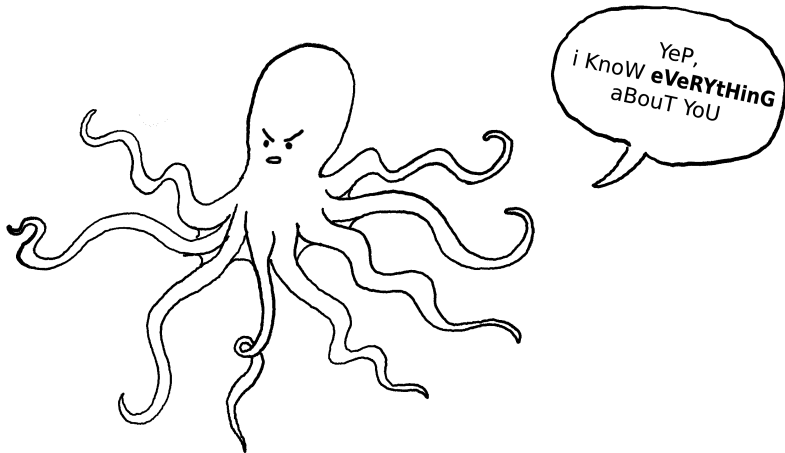


AlCrypt '26 workshop — May 9th, 2026

A story - the octopus



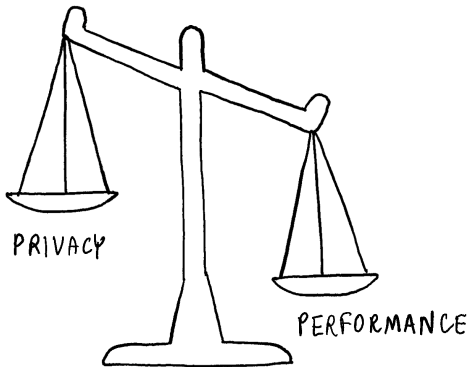
A story - the octopus



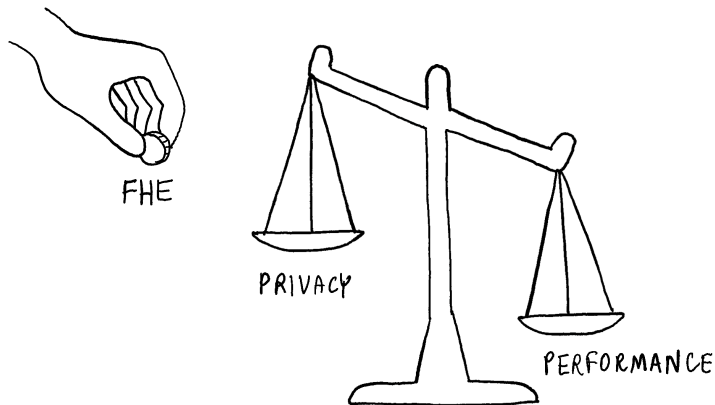
Motivations

- Nowadays recommendation systems are part of almost any digital platform (movies, songs, books, news articles, social media, jobs, dating apps...).
- If it has a catalog and users, it probably has a recommendation engine behind it.
- In order to offer such service, the provider has access to private user data.
- Somewhere, there is information about what movies we like, what songs we like, etc.
- Motivation: Recommendations are cool, but they cost our privacy!

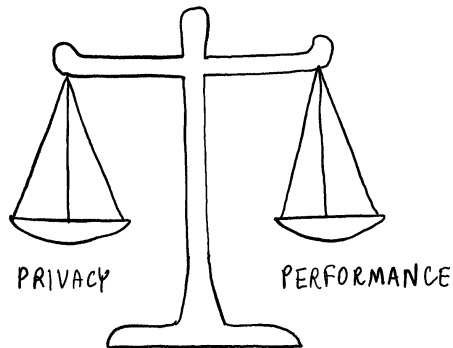
A story - the balance



A story - the balance



A story - the balance



*Hopefully :)

Motivations

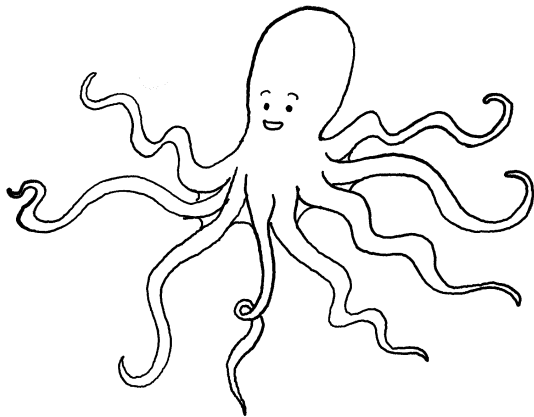
- We want to build a recommendation system based on FHE primitives
- In particular, we focus on the *retrieval* step.
- This is typically represented as

$$\operatorname{argmax}(W \cdot \mathbf{s} + \mathbf{b})$$

where W is some learnt weight matrix, $\mathbf{s}$ the user latent factor vector and $\mathbf{b}$ some bias term.

- The result of argmax is typically an index for a dict.

A story - the octopus



Some related works

- In [CBZ25], they perform a retrieval, but they return to the user the entire vector with output logits. This has several issues:
 - The result leaks strong information about $W, \mathbf{b}$ (with a linear amount of query we can recover the weights).
 - The result can not be used for further computations (e.g., retrieve some metadata about the predicted element) without an additional interaction.
- In [YL26] we have the same issue, but using a CNN: the logits are efficiently computed and returned to the user.

[CBZ25]: M. N. Chowdhury, A. Bauer, and M. Zhou. “Efficient privacy-preserving recommendation on sparse data using fully homomorphic encryption”. In: *IEEE Int. Conf. on eScience*. 2025

[YL26]: Q. Yi and R. Li. “Privacy-preserving Recommendation based on Non-interactive Encrypted CNN”. In: *CNML '26*

The goal - formally

Our goal is, given a trained (linear, for simplicity) recommender system $(W, \mathbf{b})$ and some user latent factor $\mathbf{s}$, to

1. Derive a set of logits via $W \cdot \mathbf{s} + \mathbf{b}$
2. Compute the index of the suggested element via $\text{argmax}(\cdot)$
3. Return some metadata (e.g., the name) about the element using the just recovered index (e.g., mask a dictionary with a binary mask)

For our experiments, we will use the MovieLens dataset.

The goal - formally

Our goal is, given a trained (linear, for simplicity) recommender system $(W, \mathbf{b})$ and some user latent factor $\mathbf{s}$, to

1. Derive a set of logits via $W \cdot \mathbf{s} + \mathbf{b}$ — **Easy**
2. Compute the index of the suggested element via $\text{argmax}(\cdot)$
3. Return some metadata (e.g., the name) about the element using the just recovered index (e.g., mask a dictionary with a binary mask)

For our experiments, we will use the MovieLens dataset.

The goal - formally

Our goal is, given a trained (linear, for simplicity) recommender system (W, b) and some user latent factor s , to

1. Derive a set of logits via $W \cdot s + b$ — **Easy**
2. Compute the index of the suggested element via $\text{argmax}(\cdot)$ — **Hard**
3. Return some metadata (e.g., the name) about the element using the just recovered index (e.g., mask a dictionary with a binary mask)

For our experiments, we will use the MovieLens dataset.

The goal - formally

Our goal is, given a trained (linear, for simplicity) recommender system (W, b) and some user latent factor s , to

1. Derive a set of logits via $W \cdot s + b$ — **Easy**
2. Compute the index of the suggested element via $\text{argmax}(\cdot)$ — **Hard**
3. Return some metadata (e.g., the name) about the element using the just recovered index (e.g., mask a dictionary with a binary mask) — **Medium**

For our experiments, we will use the MovieLens dataset.

The goal - formally

In this talk we will mostly focus on the second challenge:

- Compute the index of the suggested element via $\text{argmax}(\cdot)$ — **Hard**.

The goal - formally

In this talk we will mostly focus on the second challenge:

- Compute the index of the suggested element via $\text{argmax}(\cdot)$ — **Hard**.
- Our main ingredient will be an FHE-based sorting algorithm: we start from the original approach introduced in [MEHP25] and improved in [RLB26].

[MEHP25]: F. Mazzone, M. Everts, F. Hahn, and A. Peter. “Efficient Ranking, Order Statistics, and Sorting under CKKS”. In: *USENIX Security '25*

[RLB26]: L. Rovida, A. Leporati, and S. Basile. “Lightweight sorting in approximate homomorphic encryption”. In: *IACR Communications in Cryptology* (2026)

Permutation-based sorting

This approach, introduced in [MEHP25], consists of two phases:

1. Indexing: given $\mathbf{v}$, we want to build its indexing vector $\mathbf{s}$, namely the position of each element v_i in $\text{sort}(\mathbf{v})$, e.g.:

Given $\mathbf{v} = (5, 0, 9, 7)$, we get $\mathbf{s} = (2, 1, 4, 3)$

Permutation-based sorting

This approach, introduced in [MEHP25], consists of two phases:

1. Indexing: given $\mathbf{v}$, we want to build its indexing vector $\mathbf{s}$, namely the position of each element v_i in $\text{sort}(\mathbf{v})$, e.g.:

Given $\mathbf{v} = (5, 0, 9, 7)$, we get $\mathbf{s} = (2, 1, 4, 3)$

2. Permutation: given the indexing vector, we want to construct the permutation matrix P , e.g.:

Given $\mathbf{s} = (2, 1, 4, 3)$, we get $P = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$

Phase 1: Indexing

Given $\mathbf{v} \in \mathbb{R}^n$, we can recover the indexing $\mathbf{s} \in \mathbb{Z}^n$ by *counting* for each element v_i , how many elements v_j are smaller than v_i :

$$s_i = 0.5 + \sum_{j=1}^n \text{sgn}(v_i - v_j)$$

where

$$\text{sgn}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0.5 & \text{if } x = 0 \\ 0 & \text{otherwise} \end{cases}$$

Phase 1: Indexing

For instance, following the previous example with $v = (5, 0, 9, 7)$, we obtain $s = (s_1, s_2, s_3, s_4)$ such that:

$$s_1 = 0.5 + \text{sgn}(5 - 5) + \text{sgn}(5 - 0) + \text{sgn}(5 - 9) + \text{sgn}(5 - 7) = 2$$

$$s_2 = 0.5 + \text{sgn}(0 - 5) + \text{sgn}(0 - 0) + \text{sgn}(0 - 9) + \text{sgn}(0 - 7) = 1$$

$$s_3 = 0.5 + \text{sgn}(9 - 5) + \text{sgn}(9 - 0) + \text{sgn}(9 - 9) + \text{sgn}(9 - 7) = 3$$

$$s_4 = 0.5 + \text{sgn}(7 - 5) + \text{sgn}(7 - 0) + \text{sgn}(7 - 9) + \text{sgn}(7 - 7) = 4$$

We can parallelize this computations with SIMD by using $\Theta(n^2)$ slots!

Phase 2: Permutation matrix

Now compare each s_i with values from 1 up to n ! Given:

$$\text{equal}(x) = \begin{cases} 1 & \text{if } x = 0 \\ 0 & \text{otherwise} \end{cases}$$

In order to obtain the i -th column p_i of P , we simply compute¹:

$$p_i = \text{equal}((s_1, s_2, s_3, s_4) - (i, i, \dots, i))$$

The first column $i = 1$ in our example will be:

$$\begin{aligned} p_1 &= \text{equal}((2, 1, 4, 3) - (1, 1, 1, 1)) = \text{equal}((1, 0, 3, 2)) \\ &= (0, 1, 0, 0) \end{aligned}$$

¹By abuse of notation, we allow the function to be applied on vectors by applying $\text{equal}(x)$ independently to each entry.

Handling equal values

- One may notice that this approach does not handle very well repeated elements.
- For example, the indexing s given by the input vector $v = (7, 8, 8, 1)$ will be equal to $s = (2, 3.5, 3.5, 1)$

Handling equal values

- One may notice that this approach does not handle very well repeated elements.
- For example, the indexing s given by the input vector $v = (7, 8, 8, 1)$ will be equal to $s = (2, 3.5, 3.5, 1)$
- Good news: we can easily fix this!
- Idea: compute a *tie-correction offset* o . Following the example with s , we would need $o = (0, -0.5, 0.5, 0)$.

Let's make it concrete

- The presented framework abstracts from any particular computational method.
- But it well fits SIMD FHE schemes, because of its “horizontal” nature.
- Most of computations can be parallelized by repeating the values in $\mathbf{v}$, with the main obstacles being are the two nonlinear functions $\text{sgn}(x)$ and $\text{equal}(x)$.

Approximating $\text{sgn}(x)$

However, we should be cautious... $\text{sgn}(x)$ is far from being an easy function!

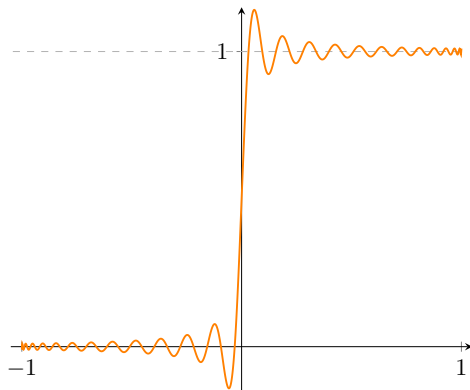


Figure: Chebyshev approximation with degree $d = 50$

Approximating $\text{sgn}(x)$

In order to understand why this happens, look at the positions of the regression points!

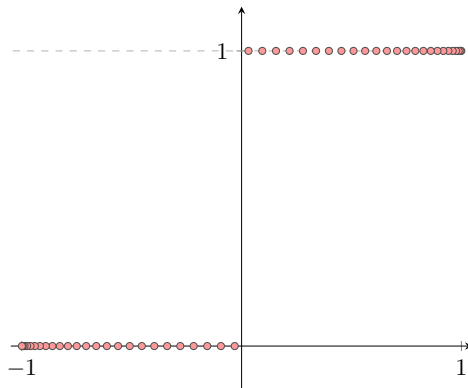


Figure: Regression points when $d = 50$

Approximating $\text{sgn}(x)$

Idea: let us try to smooth the function, use instead $1/(1 + e^{-kx})$, for a sufficiently large $k > 0$ (i.e., the sigmoid function):

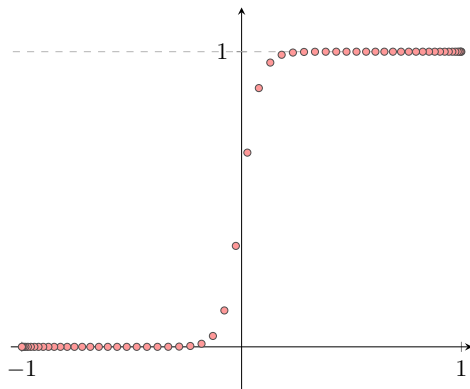


Figure: Points of sigmoid approximation, with $k = 25$ and degree $d = 50$

Approximating $\text{sgn}(x)$

This leads to an approximation that has no oscillations!

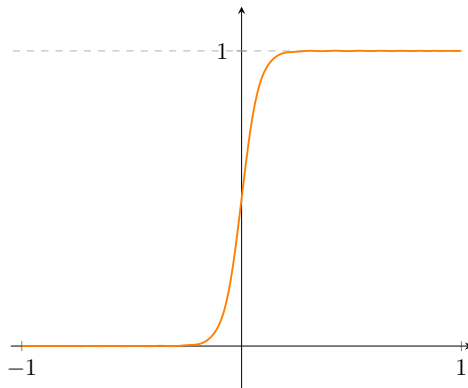


Figure: Sigmoid approximation, with $k = 25$ and degree $d = 50$

Approximating $\text{sgn}(x)$

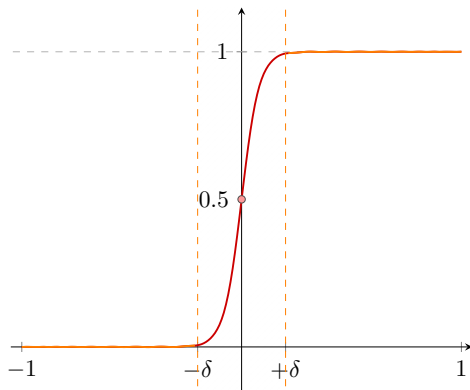
- However, this approximation does not work well when approaching the origin $x = 0$.
- We assume that every pair of element to be sorted is at distance at most δ , e.g., $\delta = 0.001$ for inputs in $[0, 1]$.
- We can fix some appropriate pair (deg, k) s.t. the corresponding polynomial well-approximates $\text{sgn}(x)$ in the following interval²:

$$[-1, -\delta] \cup [0] \cup [\delta, 1]$$

²We can prove that $x = 0$ is still approximated correctly as 0.5 for any choice of $\delta > 0, k > 1$ and $d > 1$

Approximating $\text{sgn}(x)$

Visually, if our assumptions on inputs hold, we obtain the following:



Approximating $\text{equal}(x)$

The next function is $\text{equal}(x)$, used to check equalities in the indexing vector (i.e., between approximate integers), which is even worse!

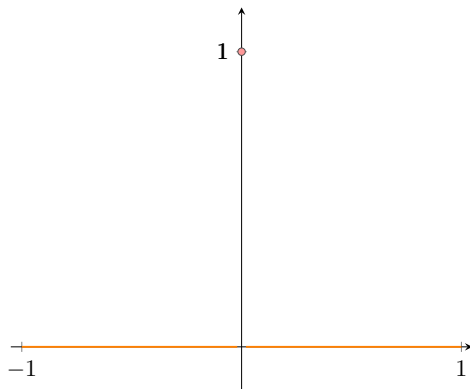


Figure: Plot of the $\text{equal}(x)$ function

Approximating $\text{equal}(x)$

In [MEHP25], they propose to use the following, which even allows to handle errors up to ± 0.5 :

$$\text{sgn}(x + 0.5/a) \cdot (1 - \text{sgn}(x - 0.5/a))$$

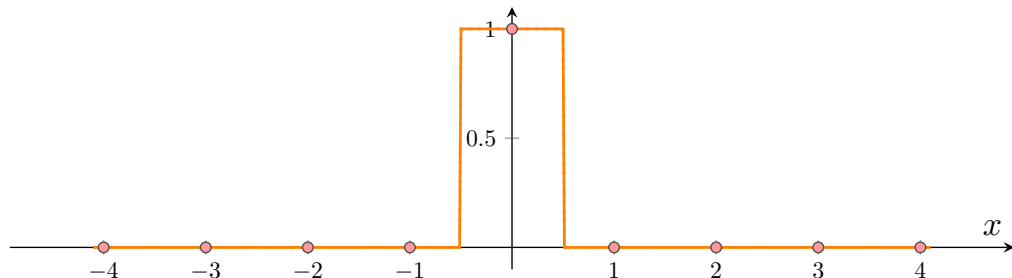


Figure: Plot of approximation of $\text{equal}(x)$ function used in [MEHP25]

Approximating $\text{equal}(x)$

Idea: let us "relax" this function by observing that we only require this two constraints:

1. $f(0) = 1$ and
2. $f(x) = 0$ for all $x \in \mathbb{Z} \setminus 0$.

Lemma ([RLB26])

There exists an algorithm that, given any function $f : \mathbb{R} \rightarrow \mathbb{R}$ such that $f(0) = 1$ and $f(x) = 0$ for all $x \in \mathbb{Z} \setminus \{0\}$, and the indexing s of some vector v , returns a permutation matrix P such that $P \cdot v$ is sorted.

Approximating $\text{equal}(x)$

A nice function that respects these constraints is the $\text{sinc}(x)$ function:

$$\text{sinc}(x) = \frac{\sin(x\pi)}{x\pi}$$

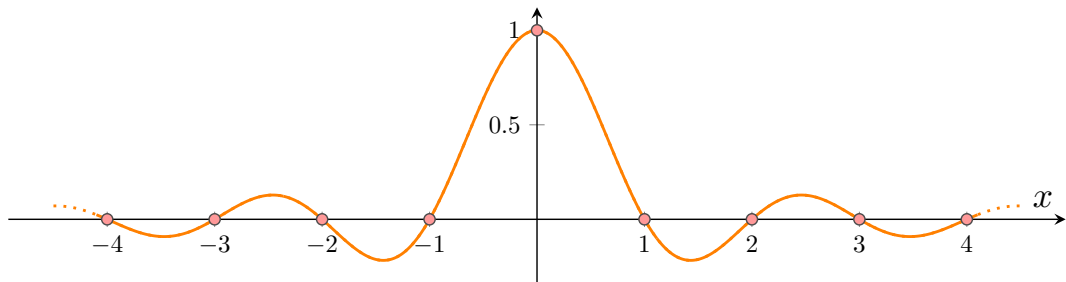


Figure: Plot of the $\text{sinc}(x)$ function

Approximating $\text{equal}(x)$

- This function can be approximated without relying on $\text{sgn}(x)$
- Even with small degrees, Chebyshev polynomials allow for very accurate approximations. It also has some “cleaning” properties!

Lemma (Cleaning properties of $\text{sinc}(x)$ [RLB26])

For every $x \in \mathbb{Z} \setminus \{\pm 1\}$ and every small $\varepsilon \in \mathbb{R}$ such that $|\varepsilon| < 1$, we have that

$$\begin{cases} |\text{sinc}(x + \varepsilon)| \leq |\varepsilon| & \text{if } x \neq 0, \\ |\text{sinc}(\varepsilon)| \leq |1 + \varepsilon| & \text{if } x = 0. \end{cases}$$

That is, the output error is reduced except in the cases of $x = \pm 1$. In particular, we have that:

$$\text{sinc}(x + \varepsilon) \approx \varepsilon/x.$$

An SIMD example

- Given the input $\mathbf{v} = (5, 0, 9, 7)$, we define two different encodings $\mathbf{v}_{\text{repeated}}$, $\mathbf{v}_{\text{expanded}}$ and compute their difference:

$$\begin{array}{lcl}
 \mathbf{v}_{\text{repeated}} : & \boxed{5} \boxed{0} \boxed{9} \boxed{7} \boxed{5} \boxed{0} \boxed{9} \boxed{7} \boxed{5} \boxed{0} \boxed{9} \boxed{7} \boxed{5} \boxed{0} \boxed{9} \boxed{7} & - \\
 \mathbf{v}_{\text{expanded}} : & \boxed{5} \boxed{5} \boxed{5} \boxed{5} \boxed{0} \boxed{0} \boxed{0} \boxed{0} \boxed{9} \boxed{9} \boxed{9} \boxed{9} \boxed{7} \boxed{7} \boxed{7} \boxed{7} & = \\
 & \hline
 \mathbf{v}_{\Delta} : & \boxed{0} \boxed{-5} \boxed{4} \boxed{2} \boxed{5} \boxed{0} \boxed{9} \boxed{7} \boxed{-4} \boxed{-9} \boxed{0} \boxed{-2} \boxed{-2} \boxed{-7} \boxed{2} \boxed{0} &
 \end{array}$$

An SIMD example

- Now we apply (an approximation of) the $\text{sgn}(x)$ function over $\mathbf{v}_\Delta$.

$\text{sgn}(\mathbf{c}_\Delta) :$

0.5	0	1	1	1	0.5	1	1	0	0	0.5	0	0	0	1	0.5
-----	---	---	---	---	-----	---	---	---	---	-----	---	---	---	---	-----

An SIMD example

- Now we apply (an approximation of) the $\text{sgn}(x)$ function on $\mathbf{v}_\Delta$.

$\text{sgn}(\mathbf{c}_\Delta) :$

0.5	0	1	1	1	0.5	1	1	0	0	0.5	0	0	0	1	0.5
-----	---	---	---	---	-----	---	---	---	---	-----	---	---	---	---	-----

An SIMD example

- Then we count how many elements in v are smaller than each v_i , using a rotate-and-sum procedure at distance n :

$\text{sgn}(c_\Delta) :$	0.5	0	1	1	1	0.5	1	1	0	0	0.5	0	0	0	1	0.5
$0.5 + \sum :$	2	1	4	3	2	1	4	3	2	1	4	3	2	1	4	3

An SIMD example

- Then we count how many elements in v are smaller than each v_i , using a rotate-and-sum procedure at distance n :

$\text{sgn}(c_\Delta) :$	0.5	0	1	1	1	0.5	1	1	0	0	0.5	0	0	0	1	0.5
$0.5 + \sum :$	2	1	4	3	2	1	4	3	2	1	4	3	2	1	4	3

An SIMD example

- Then we count how many elements in v are smaller than each v_i , using a rotate-and-sum procedure at distance n :

$\text{sgn}(c_\Delta) :$	0.5	0	1	1	1	0.5	1	1	0	0	0.5	0	0	0	1	0.5
$0.5 + \sum :$	2	1	4	3	2	1	4	3	2	1	4	3	2	1	4	3

An SIMD example

- Then we count how many elements in v are smaller than each v_i , using a rotate-and-sum procedure at distance n :

$\text{sgn}(\mathbf{c}_\Delta) :$	0.5	0	1	1	1	0.5	1	1	0	0	0.5	0	0	0	1	0.5
$0.5 + \sum :$	2	1	4	3	2	1	4	3	2	1	4	3	2	1	4	3

An SIMD example

- To conclude, we simply apply the $\text{sinc}(x)$ function!

s :

2	1	4	3	2	1	4	3	2	1	4	3	2	1	4	3
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1 to n :

1	1	1	1	2	2	2	2	3	3	3	3	4	4	4	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

$\text{sinc}(\Delta)$:

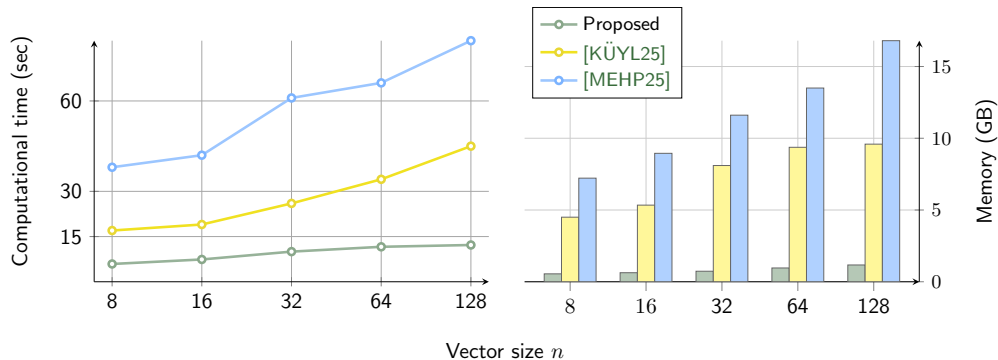
0	1	0	0	1	0	0	0	0	0	0	0	1	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

To wrap-up

- The depth of this approach is roughly given by the evaluation of $\text{sgn}(x)$ and $\text{equal}(x)$ (plus one mult to evaluate the last $P \cdot v$ product).
- Very efficient in terms of depth, might not require bootstrapping at all.
- Very inefficient in packing terms, requires n^2 slots.

To wrap-up

- However, performance are not bad.



From sorting to recommending

- This approach gives us a permutation matrix, by applying it to a set of indexes $(1, 2, \dots, n)$ we obtain the $\text{argmax}(\cdot)$ in first position.
- Issue: this method requires n^2 slots to sort a vector of size n . Our dataset (MovieLens 100k) has 1,700 entries, which would require up to 2,890,000 slots.

From sorting to recommending

- This approach gives us a permutation matrix, by applying it to a set of indexes $(1, 2, \dots, n)$ we obtain the $\text{argmax}(\cdot)$ in first position.
- Issue: this method requires n^2 slots to sort a vector of size n . Our dataset (MovieLens 100k) has 1,700 entries, which would require up to 2,890,000 slots.

Lemma

Let $b_1, b_2, \dots, b_m$ be a partition of a set s and let $k \in \mathbb{Z}_{>0}$. Then, the top- k elements of s are contained in the union of the top- k of all the b_i .

From sorting to recommending

- We fix $n = 128$ and $m = 14$ to obtain $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_{14}$, with each $\mathbf{b}_i \in \mathbb{R}^{128}$.
- Following the previous lemma, we mask the first k element from each partial sort($\mathbf{b}_i$).
- Then, we pack the $k \cdot m$ elements in a single ciphertext (we can use $k = 1$), and sort it. The final result will lie in the first slot.

Conclusions and open questions

- We presented a purely proof-of-concept to explore this idea, although in practice CKKS parameters are very large and might require bootstrapping
- This work is still under construction and improvements/tricks/suggestions are very welcome

 [Lightweight-Sorting-In-Approximate-Homomorphic-Encryption](#)  Public

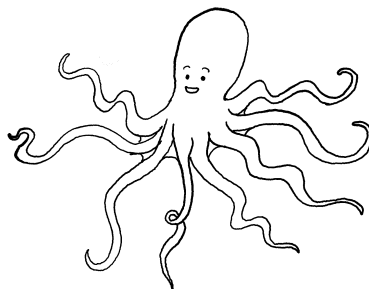
Source code for the paper "Lightweight Sorting in Approximate Homomorphic Encryption"

 Jupyter Notebook  7  2



Conclusions and open questions

Thank you! Questions?^{5,6}



⁵(Real) Octopuses have one central brain plus a mini-brain in each arm that can act independently, so their arms can *literally* think for themselves. They're considered among the smartest invertebrates on Earth. They can solve puzzles, open jars, navigate mazes and even recognize individual human faces. They are cool (don't eat 'em:))

⁶Drawing credits: F.V.

References I

- [CBZ25] M. N. Chowdhury, A. Bauer, and M. Zhou. “Efficient privacy-preserving recommendation on sparse data using fully homomorphic encryption”. In: *IEEE Int. Conf. on eScience*. 2025.
- [MEHP25] F. Mazzone, M. Everts, F. Hahn, and A. Peter. “Efficient Ranking, Order Statistics, and Sorting under CKKS”. In: *USENIX Security '25*.
- [RLB26] L. Rovida, A. Leporati, and S. Basile. “Lightweight sorting in approximate homomorphic encryption”. In: *IACR Communications in Cryptology* (2026).
- [YL26] Q. Yi and R. Li. “Privacy-preserving Recommendation based on Non-interactive Encrypted CNN”. In: *CNML '26*.